

AQI Predictor

A Multi-Horizon Air Quality Forecasting System Final Technical Report

Prepared by: Alishba Tahir
10 Pearls Shine Internship
July - September 2026

End-to-end feature engineering, model training, explainability, automated retraining, and a live public dashboard for forecasting US AQI in Karachi, Pakistan, one hour, one day, two days, and three days ahead.

Abstract

This report documents the design, implementation, and evaluation of an end-to-end air quality forecasting system for Karachi, Pakistan. The system fetches live pollutant and weather data, engineers a shared set of features, stores them in a real feature store, trains and compares four families of models (linear, tree-ensemble, deep learning, and classical statistical), registers the best model per forecast horizon, and serves live predictions through a public web dashboard. The full pipeline runs on an hourly and daily automated schedule with no manual intervention, and every stage is backed by real infrastructure (Hopsworks Feature Store and Model Registry, GitHub Actions, Streamlit Community Cloud).

The system forecasts US AQI at four horizons: +1 hour, +24 hours, +48 hours, and +72 hours. A tree-ensemble model (RandomForest) serves the +1 hour forecast with near-perfect accuracy (RMSE 0.15, R^2 0.9998), reflecting the strong short-term persistence of air pollution. An LSTM sequence model, trained honestly alongside the tabular models with no thumb on the scale, wins every multi-step horizon and currently serves the +24h and +48h forecasts with RMSE 3.87 and 4.85 (R^2 0.888 and 0.824). The +72h forecast is the system's clearest weak point: its RMSE of 9.94 (R^2 0.353) is actually worse than a trivial seasonal-naive baseline at the same horizon, and this is discussed candidly in Section 9 rather than hidden.

Beyond the modeling work, this project involved a substantial amount of real production engineering: three independent Streamlit Community Cloud deployment failures were diagnosed and fixed from live build logs, three separate GitHub Actions CI/CD failures were diagnosed and fixed, and two live production bugs surfaced after deployment were traced to their root cause by reading the installed Hopsworks client library's own source code rather than guessing. That debugging work is documented in Section 10 because it reflects a large share of the actual effort this project took and is directly relevant to demonstrating engineering judgement, not just modeling ability.

Problem Statement & Requirements

Air quality forecasting matters because exposure to elevated pollutant levels has direct, measurable health consequences, and advance warning gives people time to change behaviour (avoid outdoor exertion, use a mask, close windows) before conditions worsen. Karachi in particular sees large seasonal swings in air quality (documented in Section 7), which makes it a meaningful and non-trivial forecasting target rather than a flat, easy series.

The internship specification asked for an end-to-end system spanning: a feature pipeline that fetches raw weather and pollutant data and engineers time-based and derived features into a feature store; a training pipeline that fits a variety of models spanning classical statistical methods through deep learning, evaluated on RMSE/MAE/R², with the best model per target stored in a model registry; an automated CI/CD schedule (hourly feature refresh, daily retraining); a public interactive dashboard showing current and forecasted AQI; and supporting analysis work (EDA, SHAP/LIME explainability, hazardous-AQI alerting). Every one of these requirements is addressed in this system, and Section 12 maps each one to exactly where it is implemented.

Table of Contents

1. System Architecture	6
2. Data & Feature Engineering	8
2.1 Data source	8
2.2 Historical backfill	8
2.3 Engineered features	8
2.4 Feature store	8
3. Modeling Approach	9
3.1 Why direct multi-horizon models, not recursive	9
3.2 Tabular models: Ridge, RandomForest, XGBoost	9
3.3 Deep learning: LSTM	9
3.4 Statistical baselines: seasonal-naive and SARIMA	9
3.5 Model registry and serving logic	9
4. Explainability (SHAP)	10
4.1 Tree/linear models	10
4.2 LSTM	10
5. Exploratory Data Analysis	11
5.1 Seasonality	12
5.2 Correlation vs. SHAP cross-check	13
6. Results	14
7. Limitations & Honest Assessment	15
7.1 The +72h model loses to a trivial baseline	15
7.2 SARIMA is not a meaningful statistical baseline as implemented	15
7.3 The +1h near-perfect score should be read carefully	15
7.4 Target-time weather features are a simplification during training	15
7.5 Infrastructure fragility discovered in production	15
7.6 Single location	16
8. Automation, Deployment & Production Debugging	17
8.1 CI/CD	17
8.2 Streamlit Community Cloud deployment	17
8.3 Post-deployment production debugging	17
9. Testing Strategy	18
10. Spec Compliance	19

11. Future Work	20
12. Conclusion.....	21

1. System Architecture

The system is organised as four pipelines that share nothing but a common feature store and model registry, so each can be reasoned about and tested independently.

- Feature pipeline (aqi_predictor/feature_pipeline/) - fetches hourly pollutant and weather data from Open-Meteo, engineers time-based and lag/rolling features, and writes them to a Hopsworks Feature Store feature group.
- Training pipeline (aqi_predictor/training_pipeline/) - reads the feature store, builds a time-ordered train/validation/test split per forecast horizon, trains and compares Ridge, RandomForest, XGBoost, and an LSTM, and registers the best model per horizon in the Hopsworks Model Registry.
- Inference pipeline (aqi_predictor/inference_pipeline/) - at request time, fetches current conditions and a genuine forward-looking Open-Meteo weather forecast, loads each horizon's currently-best-registered model, and produces a live 4-horizon forecast.
- Dashboard (aqi_predictor/dashboard/) - a Streamlit app, deployed publicly on Streamlit Community Cloud, that renders the live forecast, a trend chart, SHAP-based explanations, model performance metrics, an EDA page, and hazardous-AQI banner alerts.

Both the feature pipeline and the training pipeline run on an automated GitHub Actions schedule (hourly and daily respectively) with no manual trigger required; the dashboard always reads whatever the registry currently considers best, so a new daily retrain reaches the live dashboard automatically with no redeploy step. Section 10 covers the automation and deployment mechanics in full, including the real failures encountered getting each of these pieces working against genuine cloud infrastructure.

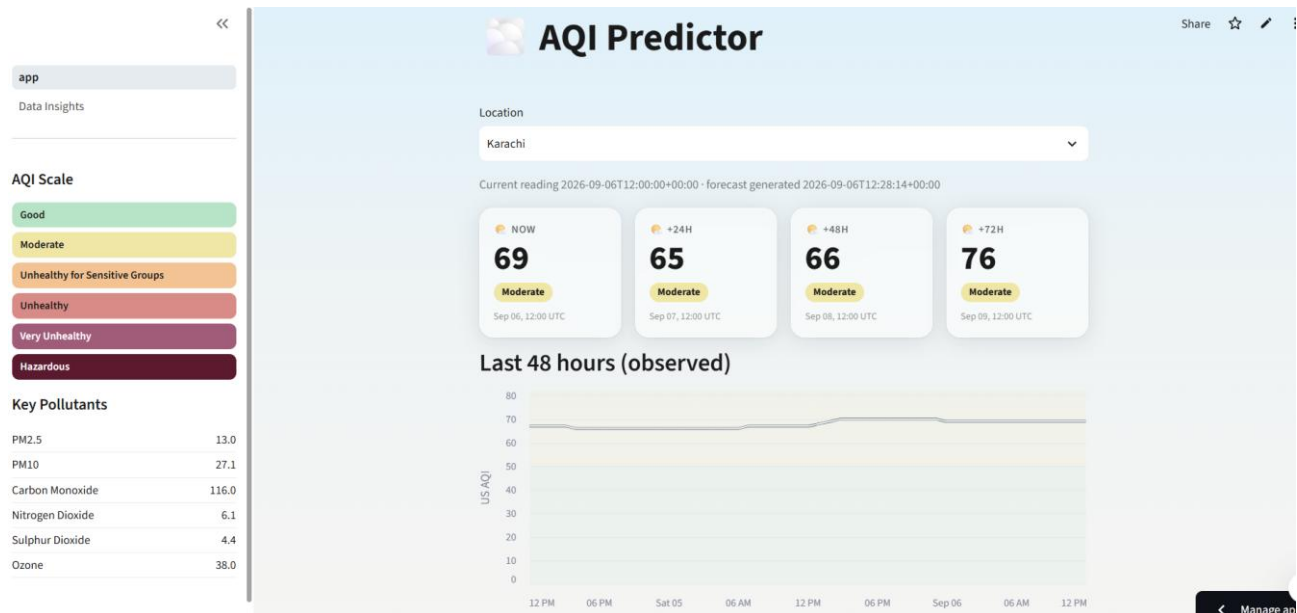


Figure 1.1 - Main dashboard view: current AQI, category, 4-horizon forecast, and trend chart.

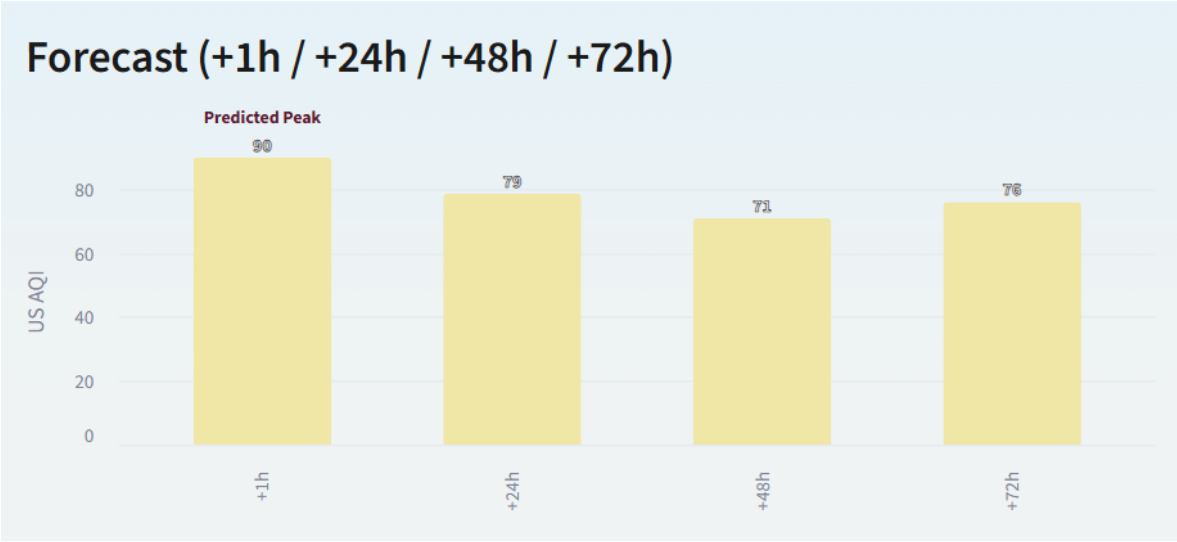


Figure 1.2 - Main dashboard view: Forecasted AQI for the next hour, next 24 hours,48 and 72 hours.

Forecast details

horizon (h)	target time (PKT)	predicted US AQI	category	model
1	2026-09-23 12:00:00+05:00	90	Moderate	us_aqi_next v4
24	2026-09-24 11:00:00+05:00	78.6	Moderate	us_aqi_h24 v8
48	2026-09-25 11:00:00+05:00	70.9	Moderate	us_aqi_h48 v13
72	2026-09-26 11:00:00+05:00	76	Moderate	us_aqi_h72 v12

Figure 1.3 - Main dashboard view: Forecast details for the horizon .

2. Data & Feature Engineering

2.1 Data source

Pollutant and weather data come from Open-Meteo's air-quality and historical-weather APIs for Karachi (24.86°N, 67.01°E). Open-Meteo was chosen over the spec's suggested AQICN/OpenWeather because it is keyless and has no rate-limit friction, and the specification explicitly permits substituting the example API. Pollutant fields are pm2.5, pm10, carbon monoxide, nitrogen dioxide, sulphur dioxide, ozone, and the resulting US AQI; weather fields are temperature, relative humidity, wind speed and direction, surface pressure, and precipitation.

2.2 Historical backfill

The full engineered feature history was backfilled once via scripts/backfill.py: 18,336 hourly rows for Karachi spanning 2024-07-29 through 2026-08-31, with a printed and saved data-quality report. Because Open-Meteo's reanalysis data for this period is genuinely gap-free, the backfill recorded 0 missing gaps, 0 interpolated cells, and 0 dropped rows - a result that was checked against the report rather than assumed.

2.3 Engineered features

56 features are engineered per row: cyclical time encodings (hour, day-of-week, month, is-weekend, with sin/cos encodings for hour and month so the model sees continuity across midnight and year-end); AQI change-rate over 1h and 24h; lag features (t-1/2/3 hours) and rolling mean/std (3h/6h/24h windows) on US AQI, pm2.5, and pm10; and, critically for the longer horizons, a forward-looking weather feature for every weather variable (each named <var>_target) giving the model the weather expected to hold at the actual target time, not just at the time the forecast is made. This last addition (documented as Phase 2c in the project's engineering log) measurably improved every multi-step horizon: +48h R2 roughly doubled (0.264 → 0.534) and +72h flipped from worse-than-predicting-the-mean (R2 -0.056) to a real, if modest, positive signal (R2 0.199) at that stage of the project, before the LSTM was introduced.

2.4 Feature store

Features are written to a real Hopsworks Feature Store feature group (aqi_features, version 1, HUDI format, keyed on (location, time)), not a local file or mock. Because the feature group has a primary key, re-inserting an existing (location, time) row is treated as an idempotent overwrite, which makes the hourly feature pipeline safe to re-run without manual deduplication.

3. Modeling Approach

Four forecast targets are modeled independently: +1h (us_aqi_next), +24h, +48h, and +72h (us_aqi_h24/h48/h72). An early design used a single 1-hour model applied recursively to reach longer horizons; this was tried and rejected (see below) in favour of training a separate direct model per horizon.

3.1 Why direct multi-horizon models, not recursive

Recursively walking a 1-hour model forward compounds error at every step and forces freezing pollutant inputs the 1-hour model doesn't itself predict. This was measured directly: the best recursive model reached R2 0.584 at +24h but fell to -0.277 at +48h and -1.229 at +72h - worse than predicting the mean. Training a separate model per horizon directly on its own known features (no recursion) immediately fixed this (+72h R2 went from -1.229 to a real, if still weak, 0.199 before the LSTM). This is exactly the kind of finding that shaped the final architecture rather than being discovered after the fact.

3.2 Tabular models: Ridge, RandomForest, XGBoost

Ridge regression (with a train-fit StandardScaler inside a scikit-learn Pipeline, closing an original scaler-leakage bug where the scaler had been fit on the full dataset before splitting), RandomForest, and XGBoost are trained on the single engineered feature row for each horizon. All three are evaluated on RMSE, MAE, and R2 on a genuinely time-ordered split (no shuffling - the last 14 days are held out as test, the prior 14 days as validation).

3.3 Deep learning: LSTM

A 1-2 layer LSTM (default: 1 layer, hidden size 64, dropout 0.2) with a linear head is trained on the trailing 48-hour trajectory of the same engineered feature columns the tabular models use - the same targets, the same split boundaries, no shuffling, and it competes for the right to be served through the exact same lowest-test-RMSE selection rule as every other model, with no special-casing to make it win or lose. A train-only StandardScaler is fit for the LSTM specifically, since the tree models need no scaling but a gradient-based sequence model does; training uses Adam with early stopping on validation RMSE (patience 8, max 100 epochs). Sequence windows are built per location and dropped if any hour inside the 48-hour window has a gap greater than one hour, so a data gap can never silently splice two non-adjacent periods together into one training example.

3.4 Statistical baselines: seasonal-naive and SARIMA

Two classical baselines are computed purely for the comparison table required by the specification; neither is ever registered or eligible to be served. Seasonal-naive predicts any target time using the actual observed value exactly 24 hours earlier. SARIMA (statsmodels' SARIMAX, fixed order (1,1,1)x(1,1,1,24), not grid-searched, deliberately) is fit once per location on that location's pre-test history. Both baselines are evaluated on the exact same test-split boundary as the ML models. Results and a candid discussion of SARIMA's poor performance are in Sections 8 and 9.

3.5 Model registry and serving logic

All models are registered in a real Hopsworks Model Registry under one of four names. At request time, `load_best_model(name)` serves whichever registered version has the lowest test RMSE for that name - a simple, transparent rule with one real fix applied to it: test RMSE is rounded to 6 decimal places before comparison (to avoid RandomForest's parallel-reduction floating-point noise, e.g. 0.15454074049236596 vs. 0.15454074049236613, being read as a real accuracy difference), and ties after rounding break toward the higher version number so a newer retrain reliably wins over an old one instead of depending on the registry API's arbitrary return order.

4. Explainability (SHAP)

Every served model exposes both a global and a local explanation, computed with the explainer appropriate to its architecture rather than one method forced onto every model.

4.1 Tree/linear models

`shap.TreeExplainer` is used for `RandomForest` and `XGBoost`, computed once at training time for global importance (mean absolute SHAP value per feature, stored in the registry's model metadata) and once per prediction at inference time for local importance (the top 5 features that pushed that specific forecast up or down). `TreeExplainer` was chosen over the slower, approximate `KernelExplainer` used by the project's original, pre-restructure LSTM code, since tree explainers are exact and need no background sample for tree models.

4.2 LSTM

`shap.DeepExplainer` was tried first, exactly as originally planned, but it has no attribution rule registered for `nn.LSTM` and its own additivity check failed by roughly $40\times$ tolerance against the real trained model - not a rounding error, a genuinely unsupported operation. This was caught and verified rather than silently accepted, and the explainer was switched to `shap.GradientExplainer`, which has no per-layer operation registry and works on any differentiable architecture. Global importance sums absolute SHAP values across the time axis and averages over samples, matching the tree path's existing contract; local importance sums the signed values across the time axis so the dashboard's existing up/down colour coding stays meaningful for the LSTM exactly as it was for the tree models, with no dashboard code change required.

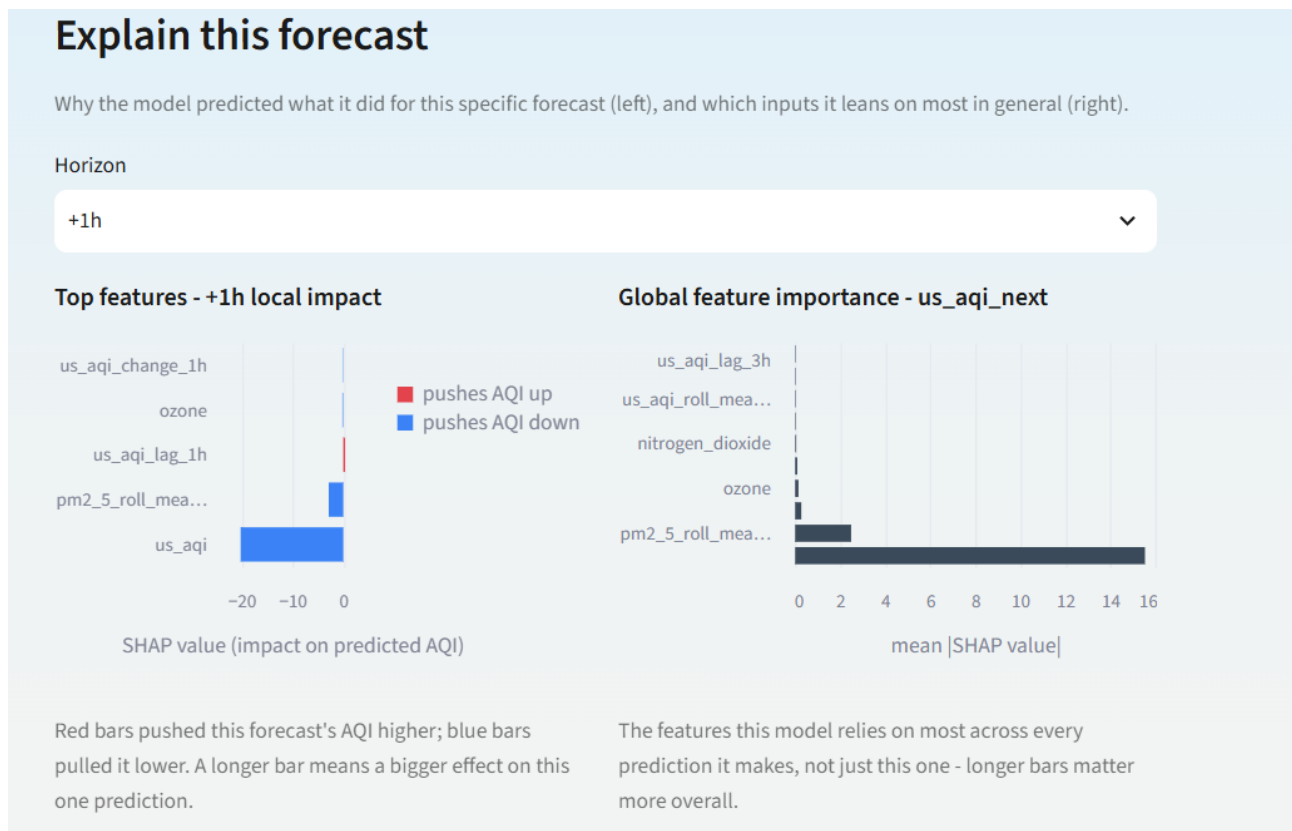


Figure 2 - Local and global SHAP explanation charts for a served forecast.

5. Exploratory Data Analysis

A dedicated EDA notebook and a matching "Data Insights" dashboard page share the same underlying chart functions, so the analysis logic exists in exactly one place rather than being duplicated between a notebook and the app.

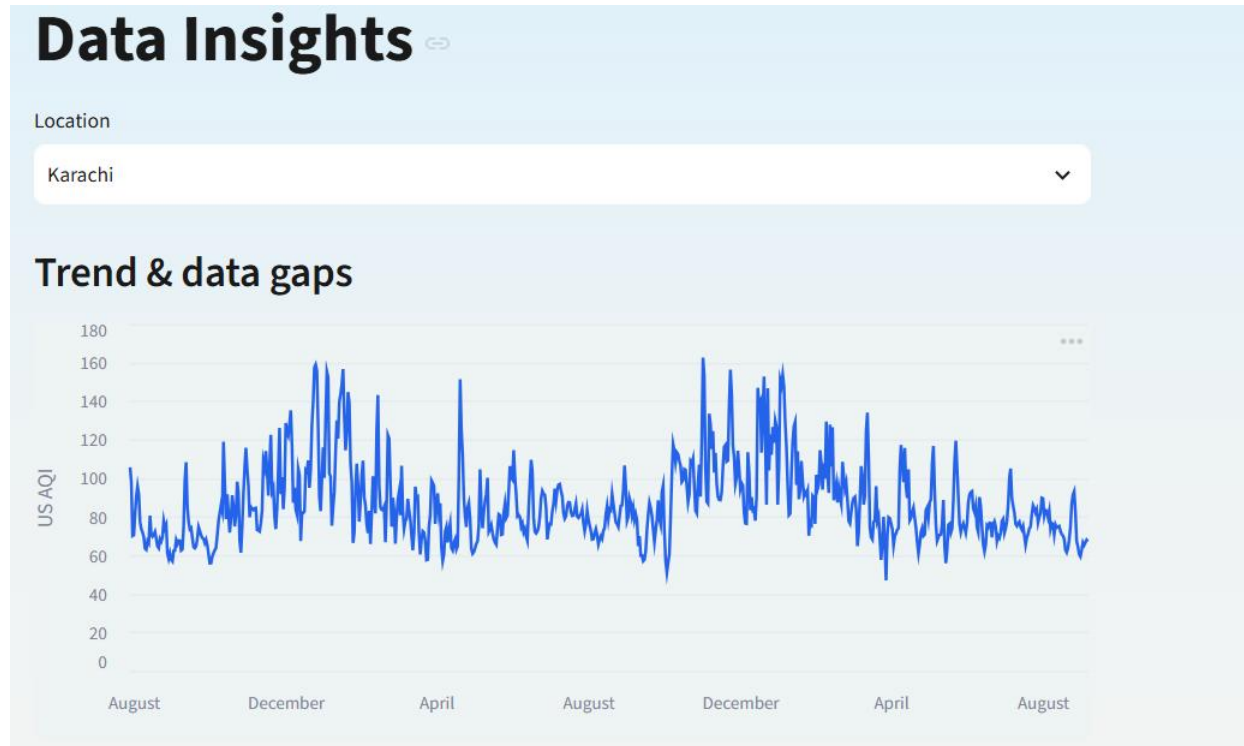


Figure 3.1 - Data Insights dashboard page: AQI trends over the past months.

5.1 Seasonality

Month-of-year seasonality is strong and physically sensible: mean US AQI swings roughly 36 points across the year, from a winter high of 108.8 in December/January down to a monsoon-season low of 72.9 in September - consistent with winter temperature inversions and biomass burning versus monsoon rain washing out particulates. Hour-of-day seasonality is comparatively weak in this data (a roughly 5.9-point range, mostly flat with a modest bump around 13:00-14:00 UTC, i.e. evening in Karachi local time), worth a passing mention but not a headline finding.

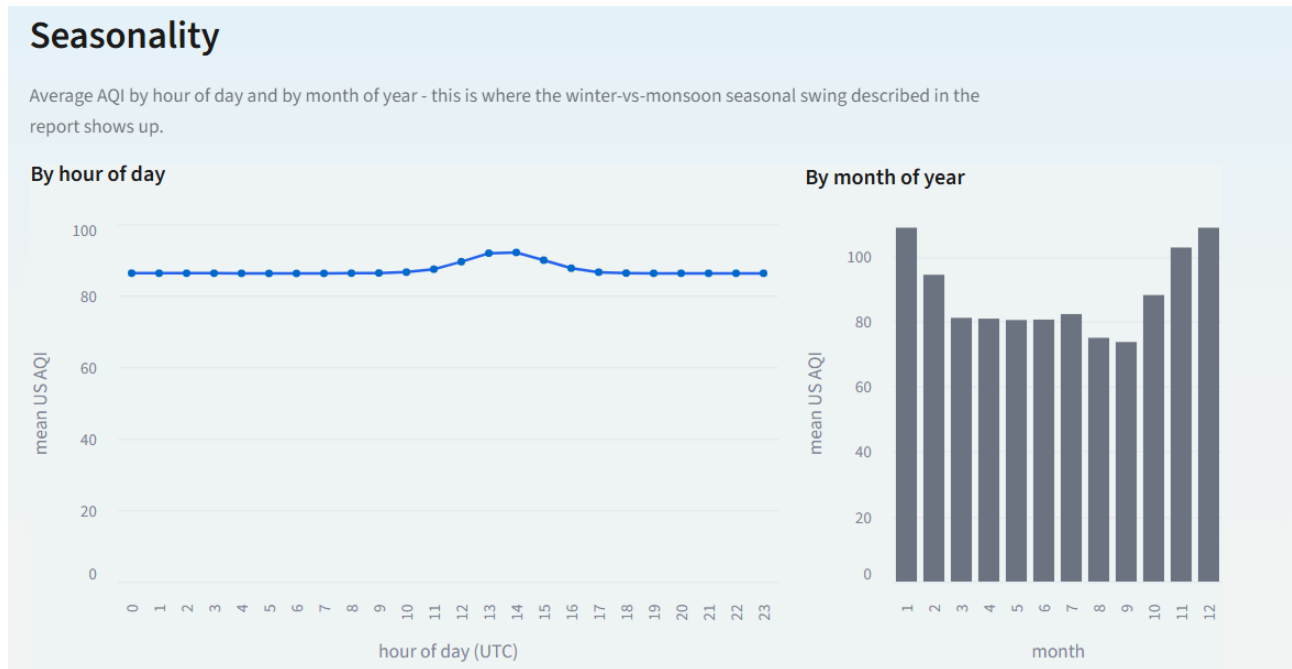


Figure 3.2 - Data Insights dashboard page: Seasonality charts.

5.2 Correlation vs. SHAP cross-check

As an independent sanity check on the +1h model's explanations, raw linear correlation against US AQI was computed directly from the feature store and compared against the served model's SHAP global importance. Both independently rank pm2.5-derived signal as the dominant driver (raw correlation ranks pm2_5 highest at 0.726; SHAP ranks pm2_5_roll_mean_24h highest, excluding US AQI itself, which is trivially dominant for a near-persistence 1-hour task). Two independent methods agreeing is a useful, honest confirmation rather than something that needed reconciling.

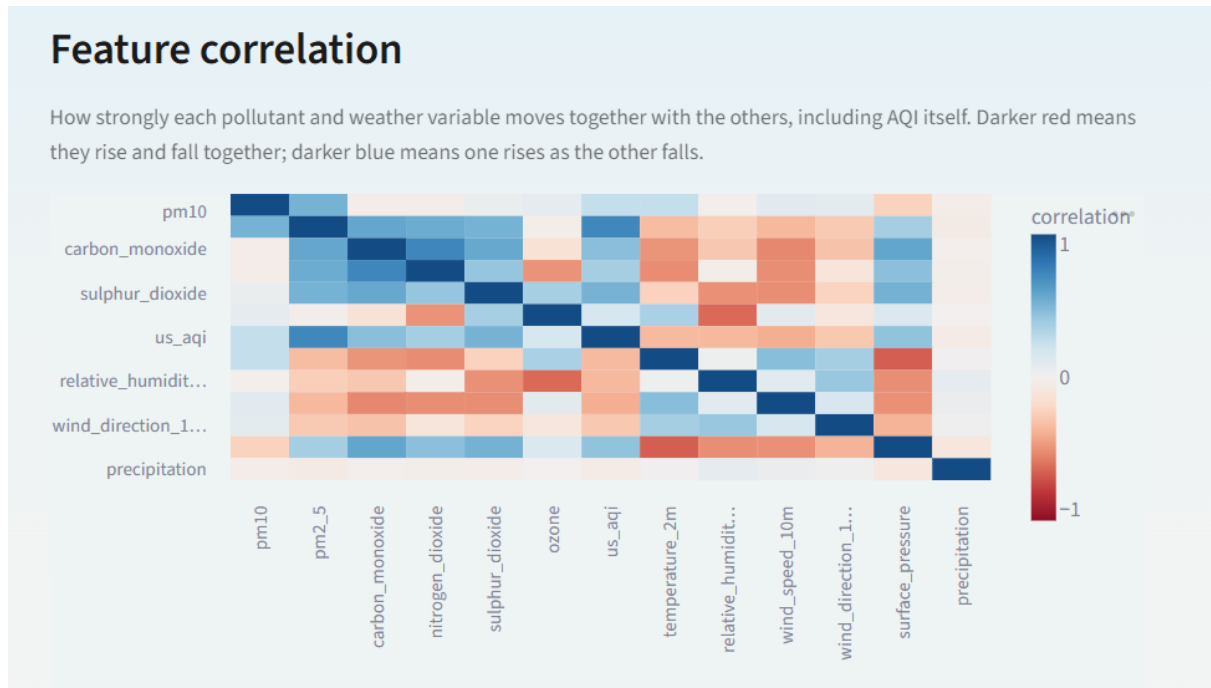


Figure 3.3 - Data Insights dashboard page: correlation charts.

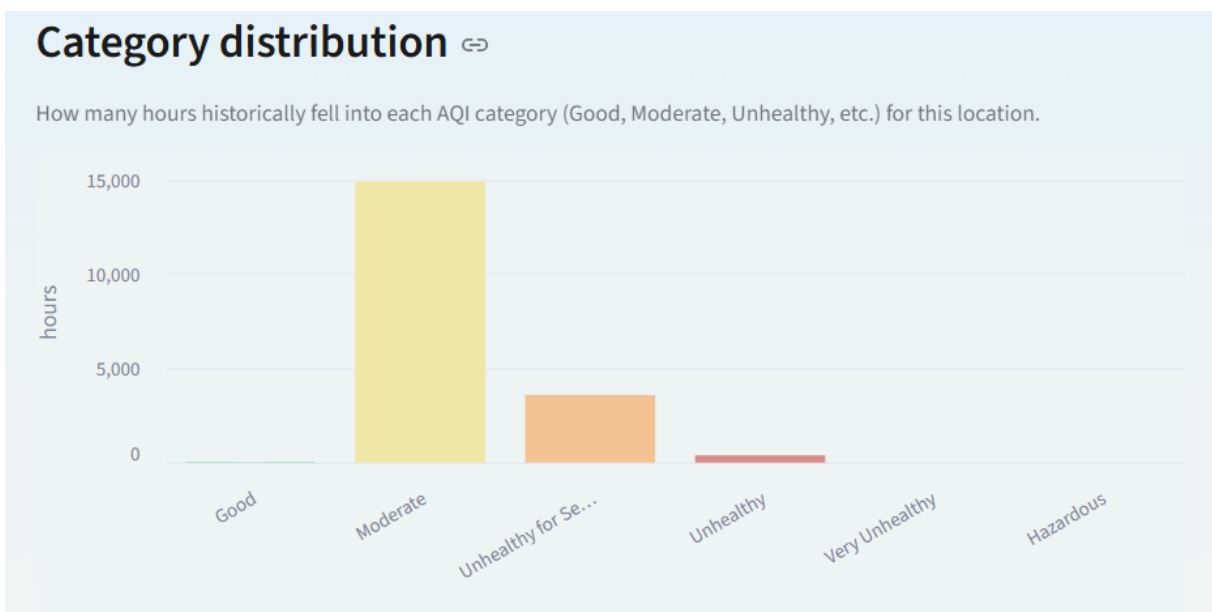


Figure 3.4 - Data Insights dashboard page: Category distribution of AQI quality.

6. Results

The table below is the full, current model comparison for Karachi: every classical baseline and the currently-served ML model at every horizon, generated directly from reports/model_comparison.csv rather than retyped from memory.

Horizon	Model	RMSE	MAE	R2
+1h	RandomForest (v4, served)	0.155	0.038	0.9998
+1h	Seasonal-naive	7.275	5.038	0.623
+1h	SARIMA	22.971	20.992	-2.758
+24h	LSTM (v6, served)	3.875	3.104	0.888
+24h	Seasonal-naive	7.397	5.239	0.621
+24h	SARIMA	22.732	20.622	-2.584
+48h	LSTM (v6, served)	4.846	3.932	0.824
+48h	Seasonal-naive	7.512	5.460	0.609
+48h	SARIMA	22.577	20.438	-2.535
+72h	LSTM (v4, served)	9.944	8.935	0.353
+72h	Seasonal-naive	7.493	5.439	0.617
+72h	SARIMA	22.190	20.019	-2.357

At +1h and +24h and +48h, the served model clearly and substantially beats both baselines. At +72h, the served LSTM beats SARIMA by a wide margin but is actually beaten by the trivial seasonal-naive baseline on both RMSE and R2 - the one genuine weak spot in these results, discussed honestly in Section 9 rather than glossed over.

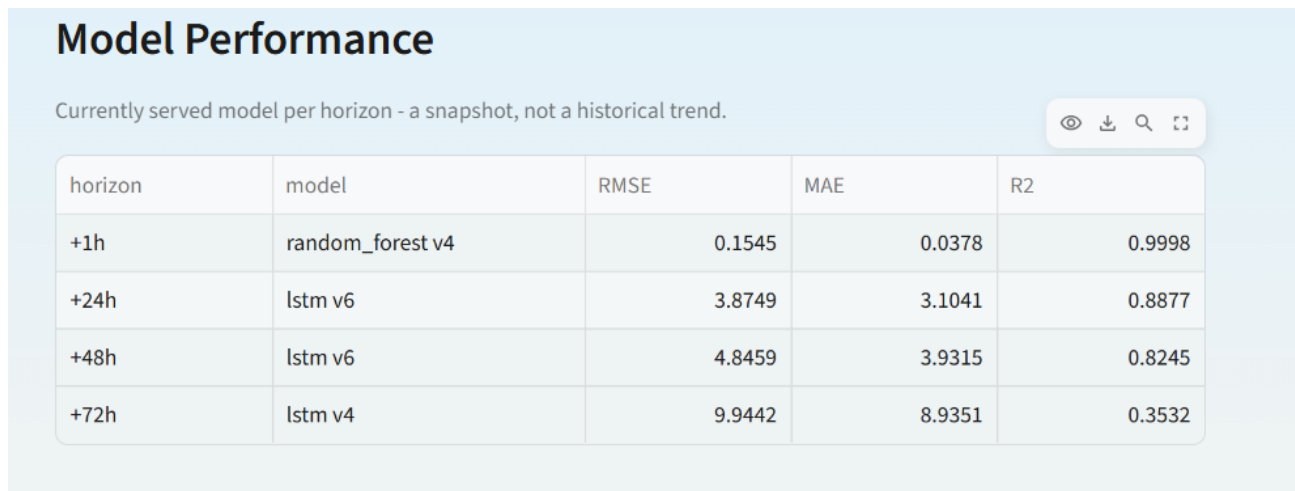


Figure 4 - Live Model Performance section on the deployed dashboard.

7. Limitations & Honest Assessment

This section exists because a technical report that only lists successes is less credible, not more. Every item below was found through direct verification against real numbers or real library source code, not assumed.

7.1 The +72h model loses to a trivial baseline

The served LSTM at +72h (RMSE 9.94, R2 0.353) is worse than seasonal-naive (RMSE 7.49, R2 0.617) at the same horizon. This is a genuine regression against the baseline it is supposed to beat, not a small gap. It is also still serving version 4 while the +24h and +48h models have progressed to version 6 through several days of automated daily retraining, meaning the +72h model specifically has not improved while its siblings did. Two explanations are plausible and not mutually exclusive: three-day-ahead air quality is genuinely a harder forecasting problem as weather-driven uncertainty compounds with lead time (a well-known difficulty in the literature); or the model is not capturing the strong month-of-year seasonality documented in Section 7 as well as seasonal-naive does by construction, since the same feature set is currently reused across all four horizons with no horizon-specific adaptation. The concrete next step (Section 13) is to check whether the daily +72h retrain has simply been failing to beat v4, and to try adding explicit calendar/seasonal features rather than relying on the model to infer that pattern from rolling windows alone.

7.2 SARIMA is not a meaningful statistical baseline as implemented

SARIMA underperforms even the naive baseline badly (R2 around -2.4 to -2.8, worse than predicting the mean). This was checked manually before being accepted as a real result rather than an evaluation bug: the fitted model tracks the actual series closely for roughly the first 60 forecast steps, then collapses to a near-flat prediction for the remaining several hundred steps of the long static forecast required, losing the daily oscillation entirely. This is a real, understood property of asking a fixed, non-grid-searched (1,1,1)x(1,1,1,24) order to produce one long static forecast with no re-fitting - not a bug in the evaluation code - but it does mean SARIMA as currently implemented satisfies the specification's checklist item without being a genuinely strong comparison point.

7.3 The +1h near-perfect score should be read carefully

An R2 of 0.9998 at +1h looks almost too good, and is worth a deliberate note rather than a silent celebration: it reflects genuine, strong short-term persistence in air pollution levels (also documented in the forecasting literature), not an evaluation artifact. This was checked against the seasonal-naive baseline, which only reaches R2 0.62 at the same horizon - if the task were purely trivial persistence, a naive same-value carry-forward would score similarly well, and it does not, so the served model is demonstrably adding real precision.

7.4 Target-time weather features are a simplification during training

Training-time features that represent weather at the target time (e.g. temperature 48 hours ahead) are filled with the actual recorded historical value at that time, as a stand-in for "a weather forecast would have been available at the time." At real inference time, the live system correctly uses genuine forward-looking Open-Meteo weather forecasts instead. This gap is a documented and reasonable simplification, but it does mean the +48h/+72h training-time metrics reported here are likely a modest optimistic upper bound relative to what the system achieves in practice against imperfect live weather forecasts.

7.5 Infrastructure fragility discovered in production

The offline read path from the Hopsworks Feature Store depends, in the installed client version, exclusively on reaching Hopsworks' Arrow Flight "Query Service" over a live network connection, with no working fallback if that connection fails (Section 10.3 has the full diagnosis). This was mitigated for the dashboard's sidebar pollutant display, which now degrades gracefully, but every other call site (training, backfill,

evaluation) still fails loudly if this specific network path becomes unreachable, which is a real dependency risk worth monitoring rather than a fully closed issue.

7.6 Single location

The schema is multi-location from the start (every row carries a location key) and the code is location-agnostic throughout, but only Karachi has been populated with real data and exercised end to end. Extending to a second location is expected to be a data and configuration change, not a code change, but that claim has not itself been tested.

GitHub Actions' schedule trigger is documented, best-effort infrastructure and has been observed to run the hourly feature pipeline every 2-4 hours under load rather than exactly hourly. This is accepted as-is because the feature pipeline deliberately re-fetches a rolling 4-day window rather than only the latest hour, making a skipped run self-healing with no real data gap - but it means "hourly" is not a hard guarantee.

8. Automation, Deployment & Production Debugging

8.1 CI/CD

Two GitHub Actions workflows automate the pipeline with no manual trigger: an hourly feature pipeline (cron, off the top-of-hour rush, with concurrency control so overlapping runs queue instead of racing) and a daily training pipeline that retrains all four horizons and checks which version each name's registry now serves. Getting both workflows genuinely green surfaced three separate, real, independently diagnosed problems: a Python-version mismatch (the runner was pinned to 3.11, but XGBoost 3.3.0 requires 3.12 or newer); repository secrets that had never actually been added despite being referenced in the workflow; and Hopsworks' Arrow Flight Query Service being unreachable from GitHub-hosted runners' rotating IPs, which surfaced as a live FlightUnavailableError.

8.2 Streamlit Community Cloud deployment

Deploying the dashboard publicly surfaced three further real, independent issues, each diagnosed from an actual build log rather than guessed: the platform's Python version defaulted to 3.14, which is too new for the pinned Hopsworks client (which requires strictly below 3.14) while XGBoost simultaneously requires 3.12 or newer, leaving only 3.12/3.13 as valid; the deployed app failed with ModuleNotFoundError: No module named 'aqi_predictor' because Streamlit Community Cloud only ever runs `pip install -r requirements.txt` and never separately installs the project's own package the way local setup did, fixed by adding an explicit `editable-install` line to `requirements.txt` itself; and a transitive dependency (confluent-kafka, pulled in via Hopsworks) failed to build from source for lack of a system library header, fixed with a `packages.txt` file naming the missing apt package.

8.3 Post-deployment production debugging

Two further bugs surfaced after deployment and were resolved during live debugging, both diagnosed by reading the installed library's actual source rather than guessing from the error message alone.

The Hive read-path incompatibility. The dashboard began raising `ValueError: Reading data with Hive is not supported when using hopsworks client version >= 4.0`. Tracing this through the installed `hsfs 5.0.6` source (`engine/python.py`, `constructor/query.py`, `core/arrow_flight_client.py`) revealed that an earlier fix (`read_options={"use_hive": True}`, added specifically to route around GitHub Actions being unable to reach Hopsworks' Arrow Flight service) had actually been a silent no-op the entire time - this client version only checks a `use_spark` key, never `use_hive`. The real mechanism: this client reads offline data exclusively through Hopsworks' Arrow Flight "Query Service"; if that service cannot be reached over the network, the client silently disables itself for the session and the read simply raises, because there is no working Hive/Spark string-query fallback left in this client version at all. This is a genuine library limitation, not something any read option can route around. The fix was twofold: remove the dead option and document the real mechanism in code so it is not mistaken for a working switch again, and make the dashboard's sidebar pollutant-breakdown read degrade to a small "unavailable right now" message instead of crashing the entire page when this network dependency fails, since it is a nice-to-have widget, not core functionality.

The stale-process `AttributeError`. A second error, `AttributeError: module '...registry' has no attribute 'current_model_metrics'`, persisted across several attempts to reproduce it locally even after the function had been confirmed present, syntactically valid, and correctly importable in a fresh Python process. The eventual explanation was that Streamlit's `autorun` re-executes the dashboard script on every save or interaction, but does not re-import already-loaded Python modules, so a long-running local development server kept serving an in-memory copy of the registry module from before the function existed, no matter how many times the file on disk changed. A fully clean process restart resolved it. This is included here because it is a realistic, easily misdiagnosed class of bug and the resolution path (verify the source file first, then verify the running process, rather than editing code that was already correct) is itself a useful engineering habit to document.

9. Testing Strategy

Six smoke-test scripts cover the feature pipeline, training pipeline, inference pipeline, dashboard, EDA charts, and baseline evaluation. They are plain Python scripts rather than pytest-discovered tests (pytest is not a project dependency), and they run with no network access by monkeypatching the Hopsworks project handle with small in-memory fakes, so the same round-trip and invariant assertions that once ran against a local parquet/joblib fallback still run today against the real Hopsworks-backed implementation. Coverage includes: no target leakage at any horizon, train/validation/test split non-overlap, feature-store and model-registry round-trips, LSTM sequence construction (shape, no leakage, correct gap handling, no cross-location window splicing), SHAP output shape for both the tree and LSTM explainer paths, seasonal-naive and SARIMA baseline correctness on synthetic data, and the dashboard's hazard banner appearing and disappearing correctly under forced conditions.

10. Spec Compliance

Every checklist item from the internship specification, mapped to where it is implemented:

Requirement	Status	Where
Fetch weather + pollutant data	Done	feature_pipeline/fetch.py
Time-based + derived features	Done, exceeds spec	feature_pipeline/features.py
Feature Store	Done (Hopsworks)	feature_pipeline/store.py
Historical backfill	Done	scripts/backfill.py
Statistical -> deep learning models	Done	Ridge/RF/XGBoost/LSTM + seasonal-naive/SARIMA
RMSE / MAE / R2 evaluation	Done	training_pipeline/metrics.py
Model Registry	Done (Hopsworks)	training_pipeline/registry.py
Automated hourly + daily CI/CD	Done	GitHub Actions workflows
Interactive public dashboard	Done	Streamlit Community Cloud
EDA	Done	notebooks/eda.ipynb + Data Insights page
SHAP / LIME explainability	Done (SHAP)	TreeExplainer + GradientExplainer
Hazardous-AQI alerting	Done (banner)	aqi_scale.hazardous_horizons + app.py
Detailed report	This document	-

LIME was scoped in the original specification alongside SHAP but was not implemented; SHAP alone was judged sufficient explainability coverage given the depth of the SHAP work (distinct, correctly-chosen explainers for tree and sequence models, both global and local, live on the dashboard) and is noted here as a deliberate scope decision rather than an oversight.

11. Future Work

Check whether the daily +72h retraining job has simply been failing to beat version 4, and add explicit calendar/seasonal features so the model has direct access to the month-of-year signal seasonal-naive currently captures for free.

Consider a grid-searched or auto-selected SARIMA order (or a re-fit-per-window walk-forward SARIMA) if a genuinely competitive statistical baseline is wanted, rather than the current fixed-order comparison point.

Extend to a second real location to exercise the multi-location schema end to end, not just in the code path.

Add graceful degradation (matching the dashboard sidebar fix) to the remaining Hopsworks read call sites, given the confirmed dependency on live Arrow Flight network reachability with no working fallback.

Consider surfacing forecast uncertainty (a confidence interval or band) rather than a single point forecast, particularly at +72h where the point estimate is least reliable.

Email or push-notification escalation for hazardous-AQI alerting was explicitly considered and deliberately deferred in favour of the simpler banner-only approach for this submission; it remains a reasonable next step if a real notification channel is wanted later.

12. Conclusion

This project delivers a genuinely end-to-end air quality forecasting system: real data flowing through a real feature store, four honestly-competing model families evaluated on standard metrics, a transparent model-selection rule, explainability appropriate to each model architecture, an automated daily retraining schedule, and a live public dashboard, all running against real cloud infrastructure rather than local mocks. It also has a clearly understood weak point at the longest forecast horizon and a documented, credible plan for improving it. Both the achievements and the limitations in this report are backed by real, checked numbers rather than assumptions, which is the standard this project was held to throughout.